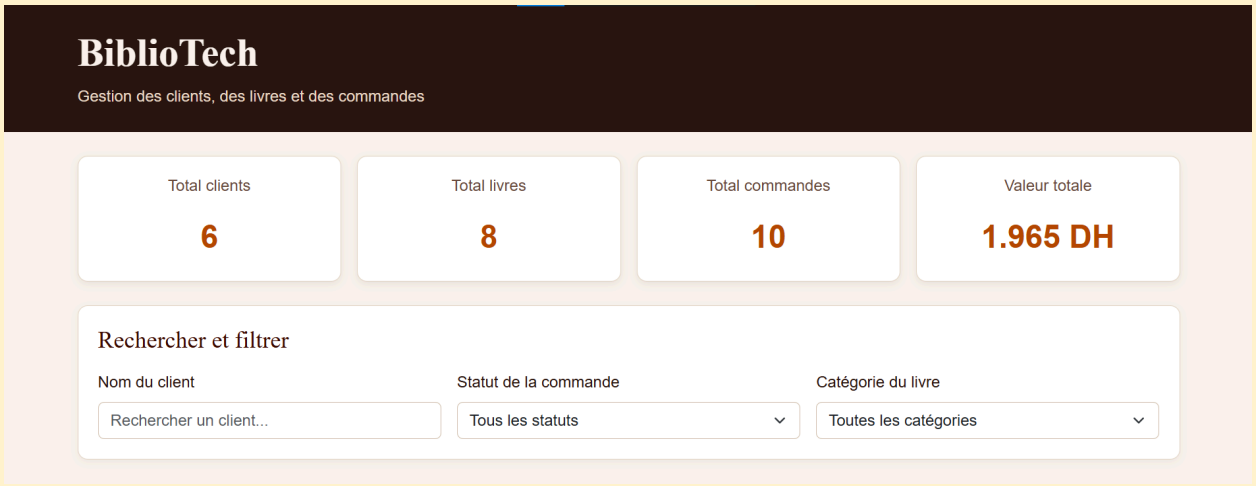
RAPPORT DE TP 4



Réalisé par : Elmoukha Fatima-ezzahra

1. Introduction

Dans le cadre de ce TP, nous avons réalisé une application web appelée **BiblioTech**, permettant de consulter et de gérer des commandes d'une boutique de livres. Ce travail nous a permis de mettre en pratique JavaScript, la manipulation des données JSON et l'utilisation de plusieurs fichiers pour organiser le code.

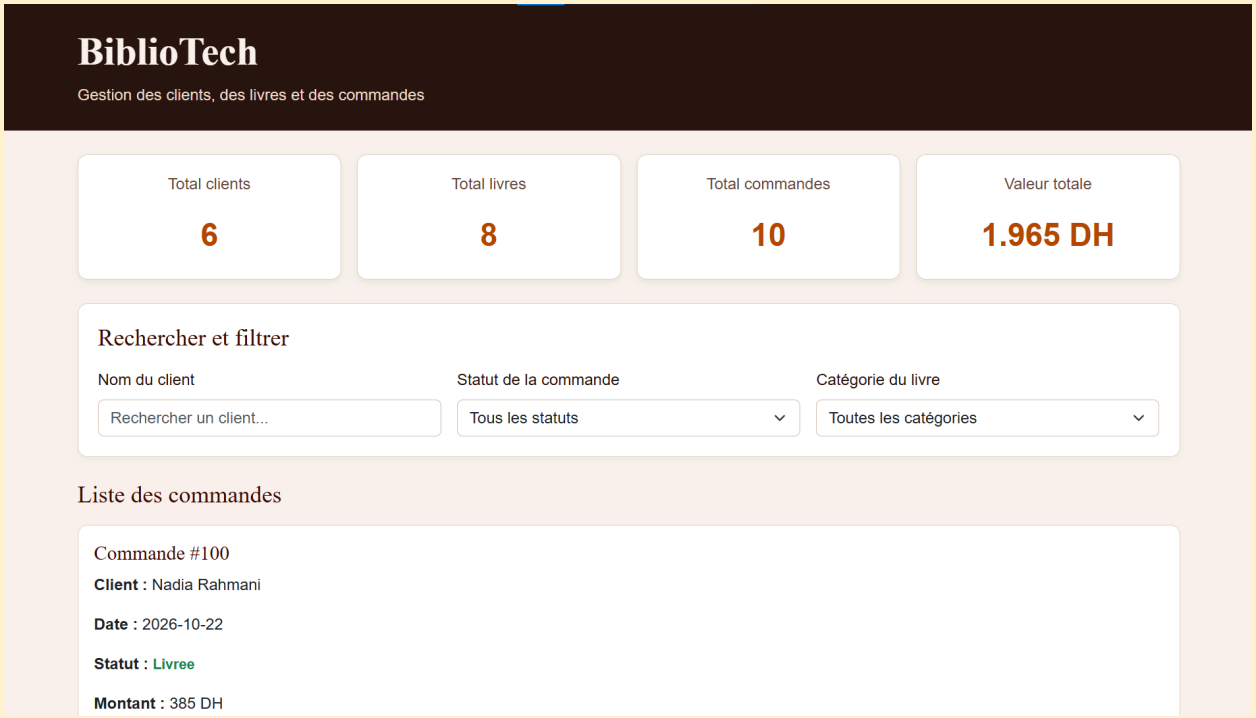
Nous avons également utilisé Git et GitHub pour suivre les modifications du projet et publier sa version finale dans un dépôt public.

2. Présentation et organisation du projet

BiblioTech est une application web qui affiche des informations sur les clients, les livres et les commandes. Les données sont enregistrées dans trois fichiers JSON séparés : `customers.json`, `products.json` et `orders.json`.

L'application permet notamment de consulter des indicateurs, de rechercher des commandes par nom du client, de filtrer les commandes selon leur statut ou la catégorie des livres, et d'afficher le détail d'une commande avec les quantités, les prix et le montant total.

Le projet est organisé en plusieurs dossiers afin de séparer la page HTML, les styles CSS, les données et les modules JavaScript.



3. Structure des fichiers JSON et relations entre les données

Dans notre projet BiblioTech, les données sont réparties dans trois fichiers JSON : `customers.json`, `products.json` et `orders.json`. Cette organisation permet de séparer les informations des clients, des livres et des commandes.

3.1. Le fichier customers.json

Ce fichier contient les informations des clients enregistrés dans l'application.

Propriété	Description
id	Identifiant unique du client

name	Nom du client
city	Ville du client
active	Indique si le client est actif

3.2. Le fichier products.json

Ce fichier contient les informations des livres disponibles dans la boutique.

Propriété	***Description***
id	Identifiant unique du livre
name	Nom du livre
category	Catégorie du livre
price	Prix du livre
stock	Quantité disponible en stock

3.3. Le fichier orders.json

Ce fichier contient les commandes effectuées par les clients. Chaque commande possède les propriétés suivantes :

Propriété	*Description*
id	Identifiant unique de la commande
customerId	Identifiant du client associé à la commande
date	Date de la commande
status	Statut de la commande
items	Liste des articles commandés

Chaque élément du tableau `items` contient deux propriétés :

- `productId` : identifiant du livre commandé.
- `quantity` : quantité commandée.

3.4. Relations entre les données

Les trois fichiers sont liés grâce aux identifiants. Dans `orders.json`, la propriété `customerId` permet de retrouver le client correspondant dans `customers.json`. De la même manière, la propriété `productId` permet de retrouver chaque livre dans `products.json`.

Ces relations évitent de répéter toutes les informations du client et du livre dans chaque commande. Le programme utilise les identifiants pour récupérer les informations nécessaires et afficher les détails d'une commande.

4. Organisation et rôle des modules JavaScript

Pour organiser le code de notre projet BiblioTech, nous avons réparti les fonctionnalités dans plusieurs fichiers JavaScript. Chaque fichier possède un rôle précis, ce qui facilite la lecture du code et la séparation des différentes tâches.

4.1. Le fichier app.js

Ce fichier représente le point de départ de l'application. Il contient la fonction `initApp()`, qui charge les données, les place dans l'état global (`state`), lance les premiers affichages et met en place les événements liés à la recherche, aux filtres et aux détails des commandes. Il gère également les erreurs pouvant survenir pendant l'initialisation.

4.2. Le fichier dataService.js

Ce fichier est responsable du chargement des données JSON. La fonction `fetchJSON()` récupère les données d'un fichier et vérifie si la réponse est correcte. La fonction `loadData()` charge les trois fichiers (`customers.json`, `products.json` et `orders.json`) et retourne les données récupérées.

4.3. Le fichier state.js

Ce fichier contient l'objet `state`, qui centralise les données de l'application : les clients, les livres et les commandes. Il contient également les valeurs utilisées pour la recherche et les filtres. Les autres modules peuvent ainsi accéder aux données et aux critères sélectionnés.

4.4. Le fichier selectors.js

Ce fichier contient des fonctions qui permettent de retrouver les clients et les livres à partir de leurs identifiants, de calculer le montant d'une commande et de filtrer les commandes selon les critères choisis. Il contient également une fonction qui calcule la valeur totale des commandes.

4.5. Le fichier view.js

Ce fichier s'occupe de l'affichage des données dans la page web. Il contient les fonctions `renderIndicators()`, `renderCategories()`, `renderOrders()` et `renderOrderDetails()`. Elles permettent respectivement d'afficher les indicateurs, les catégories disponibles, les commandes et le détail d'une commande.

Cette séparation permet de mieux organiser le projet : le chargement des données, leur traitement, leur stockage et leur affichage sont répartis dans des fichiers différents.

5. Chargement des données et gestion du `state`

5.1. Chargement des données JSON

Dans notre projet BiblioTech, les données des clients, des livres et des commandes sont stockées dans trois fichiers JSON distincts. Leur chargement est réalisé dans le fichier `dataService.js` à l'aide de `fetch()`, `async` et `await`.

La fonction `fetchJSON(url)` récupère le contenu d'un fichier JSON et vérifie la réponse grâce à la propriété `response.ok`. Si le chargement échoue, une erreur est déclenchée avec `throw new Error()`. Sinon, la méthode `response.json()` convertit les données

reçues en objets JavaScript.

La fonction `loadData()` utilise ensuite `fetchJSON()` pour charger les trois fichiers : `customers.json`, `products.json` et `orders.json`. Elle retourne les données récupérées afin qu'elles puissent être utilisées par l'application.

Le mot-clé `async` permet de définir une fonction asynchrone, tandis que `await` permet d'attendre le résultat d'une opération asynchrone avant de poursuivre l'exécution.

5.2. Gestion du state

Le fichier `state.js` contient un objet nommé `state`, qui centralise les informations utilisées par l'application. Il comprend les tableaux `customers`, `products` et `orders`, initialement vides, ainsi que les propriétés `searchText`, `statusFilter` et `categoryFilter`, qui mémorisent les critères de recherche et de filtrage.

Après le chargement des données, les résultats sont enregistrés dans les propriétés correspondantes du `state`. Les autres modules peuvent alors accéder aux données pour effectuer les calculs, appliquer les filtres et actualiser l'affichage.

5.3. Gestion des erreurs

Le chargement des données est encadré par un bloc `try...catch` dans le processus d'initialisation. Si une erreur survient, elle est interceptée et un message informe l'utilisateur que les données n'ont pas pu être chargées.

Cette gestion permet d'éviter que l'application reste sans explication lorsque l'un des fichiers JSON est indisponible. Le test de ce

fonctionnement est présenté dans la section consacrée à la gestion des erreurs.

6. Initialisation et affichage des données

6.1. La fonction d'initialisation

Dans notre projet, la fonction `initApp()` se trouve dans le fichier `app.js`. Elle coordonne le démarrage de l'application et la mise en place des événements.

Après le chargement des données, elle appelle les fonctions de rendu nécessaires pour afficher les indicateurs, les catégories et la liste initiale des commandes. Elle configure également les événements liés à la recherche, aux filtres et à la consultation des détails d'une commande.

Cette organisation permet de préparer l'application avant son utilisation et de séparer l'initialisation des autres traitements.

6.2. La fonction de rendu

Le rendu correspond à l'affichage des données dans la page HTML. Dans notre projet, les fonctions de rendu sont regroupées dans le fichier `view.js`.

- `renderIndicators()` affiche le nombre de clients, de livres et de commandes, ainsi que la valeur totale des commandes.
- `renderCategories()` récupère les catégories disponibles dans les données des livres et les ajoute à la liste des filtres.
- `renderOrders()` affiche les commandes qui correspondent aux critères de recherche et de filtrage. Chaque commande présente le client, la date, le statut, le montant et un bouton permettant de

consulter les détails.
- `renderOrderDetails()` affiche les informations détaillées d'une commande, notamment le client, sa ville, les livres commandés, les quantités, les prix unitaires et le montant total.

6.3. Mise à jour de l'affichage

Lorsque l'utilisateur saisit un nom dans le champ de recherche ou modifie un filtre, un événement est déclenché dans `app.js`. La valeur sélectionnée est mise à jour dans le `state`, puis la fonction `renderOrders()` est appelée pour actualiser la liste affichée.

Ainsi, l'affichage s'adapte aux choix de l'utilisateur sans recharger toute la page. La séparation entre les événements, le traitement des données et le rendu facilite l'organisation du code et sa maintenance.

7. Méthodes JavaScript utilisées : `find()`, `map()`, `filter()` et `forEach()`

Dans notre projet BiblioTech, nous avons utilisé plusieurs méthodes JavaScript pour rechercher des éléments dans les données, sélectionner les commandes correspondantes et afficher les informations dans la page web.

7.1. La méthode find()

La méthode `find()` permet de rechercher un élément qui correspond à une condition et de retourner le premier élément trouvé.

Dans le fichier `selectors.js`, elle est utilisée dans les fonctions `getCustomer()` et `getProduct()` pour retrouver un client ou un livre à partir de son identifiant. Elle est également utilisée dans

`view.js` pour retrouver une commande à partir de son identifiant lorsqu'on consulte ses détails.

7.2. La méthode map()

La méthode `map()` permet de créer un nouveau tableau à partir des éléments d'un tableau existant.

Dans la fonction `renderCategories()` du fichier `view.js`, elle est utilisée pour récupérer les catégories des livres à partir du tableau des produits. L'utilisation de `Set` permet ensuite de conserver une seule occurrence de chaque catégorie avant de les afficher dans la liste des filtres.

7.3. La méthode filter()

La méthode `filter()` permet de sélectionner les éléments qui respectent une ou plusieurs conditions.

Dans la fonction `getFilteredOrders()` du fichier `selectors.js`, elle permet de conserver uniquement les commandes qui correspondent aux critères choisis par l'utilisateur : le nom du client recherché, le statut de la commande et la catégorie des livres commandés.

7.4. La méthode forEach()

La méthode `forEach()` permet de parcourir les éléments d'un tableau pour effectuer une opération sur chacun d'eux.

Dans notre projet, elle est utilisée dans `renderCategories()` pour afficher les catégories, dans `renderOrders()` pour créer les cartes des commandes et dans `renderOrderDetails()` pour parcourir les

articles d'une commande et préparer les lignes du tableau détaillé.

Ces quatre méthodes facilitent le traitement des données et permettent d'organiser le code sans devoir utiliser systématiquement des boucles classiques.

8. Calculs des montants et indicateurs

8.1. Calcul du montant d'une ligne de commande

Chaque article d'une commande possède un prix unitaire, récupéré depuis `products.json`, et une quantité, enregistrée dans `orders.json`.

Le montant d'une ligne est calculé en multipliant le prix unitaire par la quantité commandée :

Montant de la ligne = Prix unitaire × Quantité

8.2. Calcul du montant total d'une commande

Une commande peut contenir plusieurs livres. Son montant total correspond à la somme des montants de toutes ses lignes.

Dans le fichier `selectors.js`, la fonction `getOrderTotal()` parcourt les articles de la commande avec `reduce()`. Pour chaque article, elle récupère le livre correspondant grâce à son identifiant, puis ajoute au total le prix du livre multiplié par la quantité commandée.

Si un livre correspondant à l'identifiant n'est pas trouvé, il n'est pas ajouté au calcul.

8.3. Calcul de la valeur totale des commandes

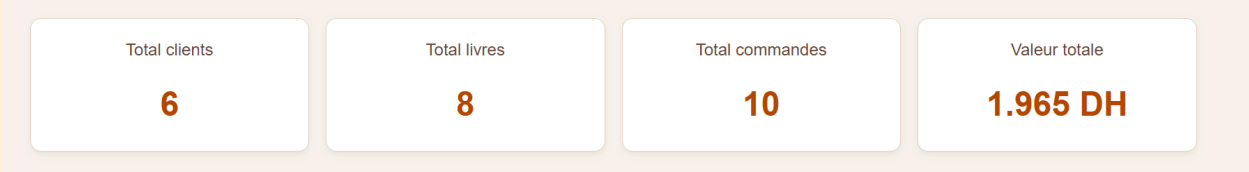
La fonction `getTotalOrderValue()` du fichier `selectors.js` calcule la somme des montants de toutes les commandes enregistrées dans le `state`. Elle utilise également `reduce()` et appelle `getOrderTotal()` pour calculer le montant de chaque commande.

8.4. Les indicateurs affichés

La fonction `renderIndicators()` du fichier `view.js` affiche quatre indicateurs sur la page principale :

- **Nombre de clients :** obtenu avec le nombre d'éléments du tableau `state.customers`.
- **Nombre de livres :** obtenu avec le nombre d'éléments du tableau `state.products`.
- **Nombre de commandes :** obtenu avec le nombre d'éléments du tableau `state.orders`.
- **Valeur totale des commandes :** calculée en additionnant les montants de toutes les commandes.

Le montant total est formaté avec `toLocaleString("fr-MA")` et affiché en dirhams marocains (DH) pour faciliter la lecture.



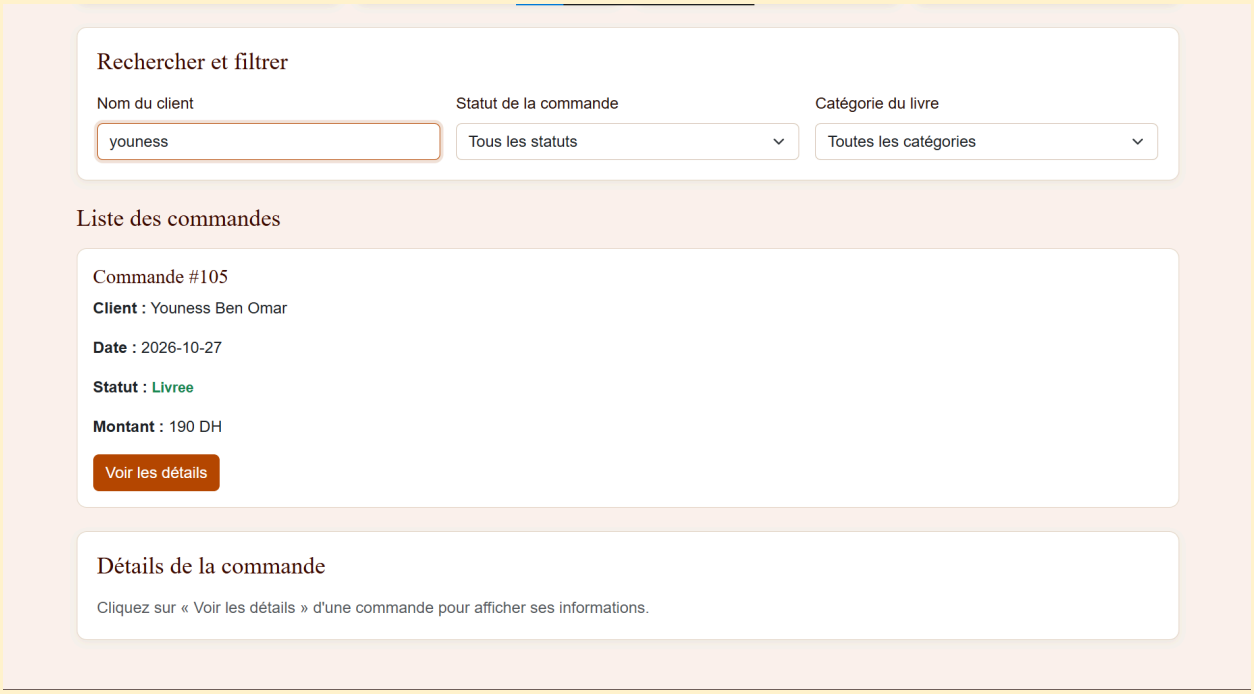
9. Recherche, filtres et détails des commandes

9.1. Recherche par nom du client

Notre application permet de rechercher une commande à partir du nom du client. Pour cela, l'utilisateur saisit un nom dans le champ de

recherche. Le programme récupère la valeur saisie, la stocke dans `state.searchText` et actualise la liste des commandes.

Dans le fichier `selectors.js`, la fonction `getFilteredOrders()` compare le texte recherché au nom de chaque client. La méthode `toLowerCase()` permet d'effectuer la comparaison sans tenir compte des majuscules et des minuscules, tandis que `includes()` vérifie si le nom contient le texte saisi.

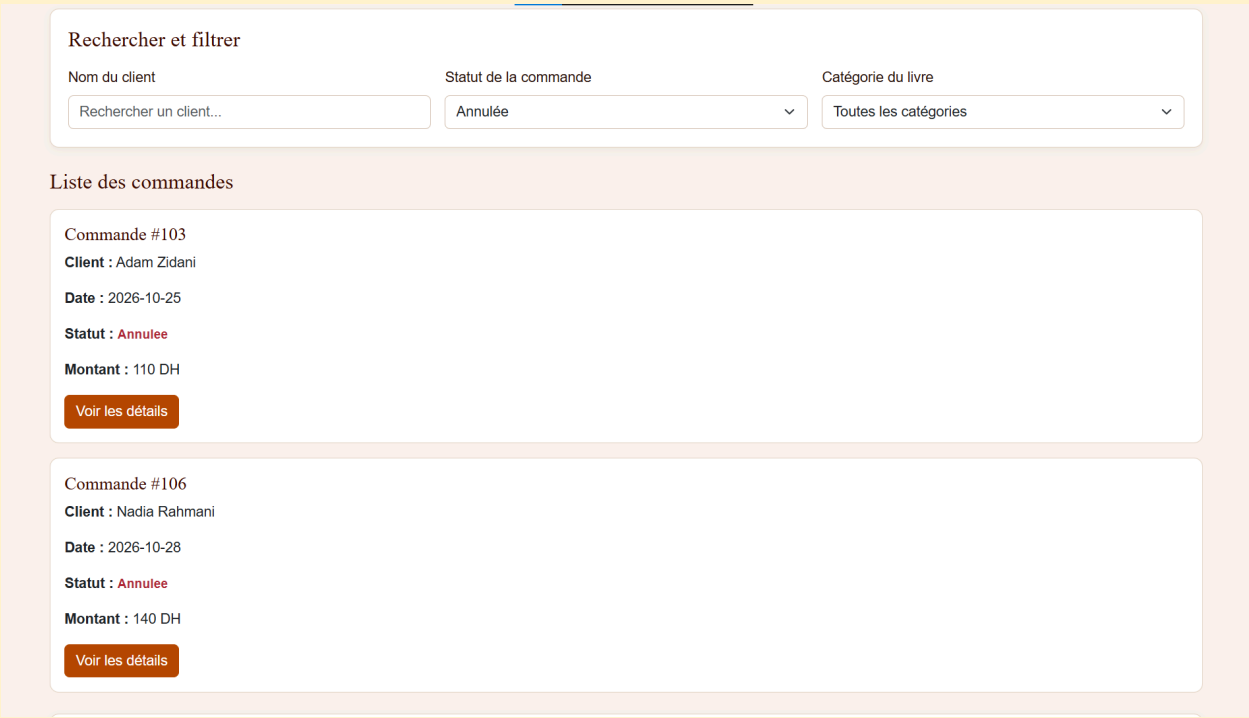


9.2. Filtrage par statut

L'utilisateur peut sélectionner un statut pour afficher les commandes correspondantes. Les statuts utilisés dans notre projet sont « Livree », « En attente » et « Annulee ». L'option « Tous » permet de ne pas limiter les résultats selon le statut.

Le statut choisi est enregistré dans `state.statusFilter`, puis utilisé dans `getFilteredOrders()` pour sélectionner les commandes

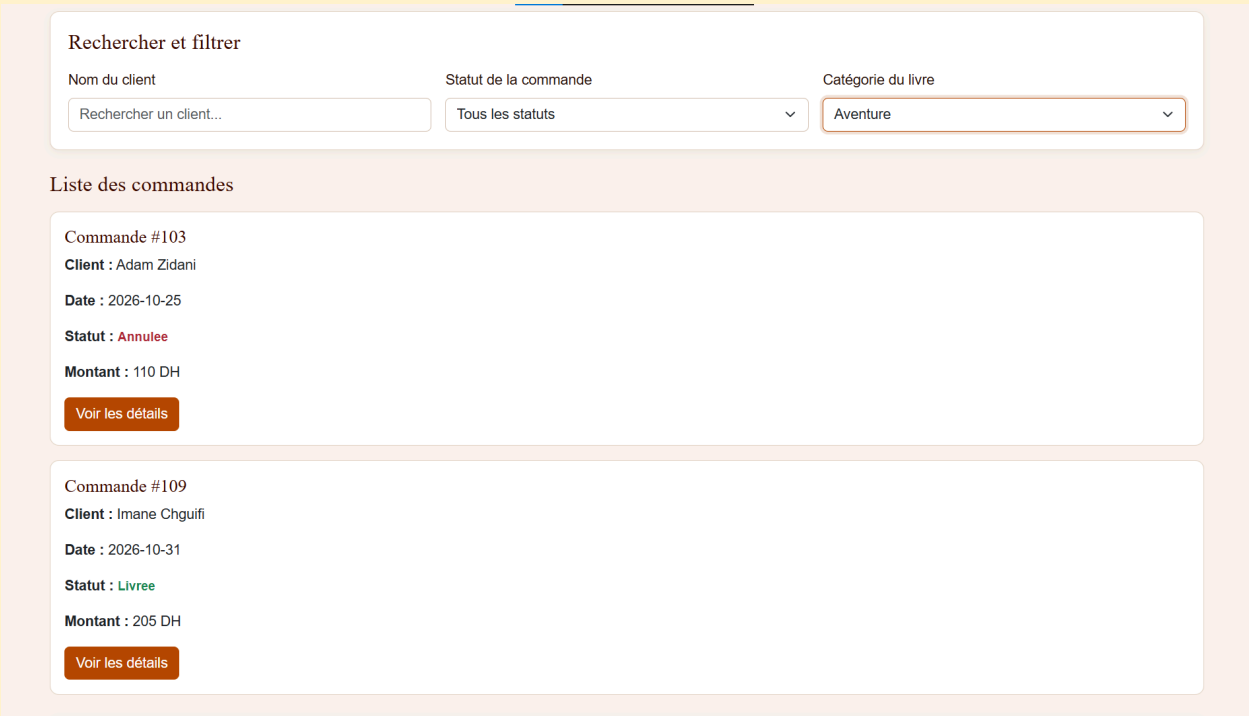
correspondantes.



9.3. Filtrage par catégorie

L'application permet également de filtrer les commandes selon la catégorie des livres qu'elles contiennent. Les catégories disponibles sont récupérées depuis `products.json` et affichées dans une liste déroulante.

Lorsqu'une catégorie est sélectionnée, le programme vérifie si au moins un article de chaque commande correspond à cette catégorie. La méthode `some()` est utilisée pour effectuer cette vérification. L'option « Toutes les catégories » permet d'afficher les commandes sans restriction de catégorie.



9.4. Combinaison de la recherche et des filtres

La recherche par nom du client, le filtre par statut et le filtre par catégorie fonctionnent ensemble. Une commande est affichée uniquement si elle respecte tous les critères actifs.



Après chaque modification, les critères sont enregistrés dans le `state` et la fonction `renderOrders()` actualise la liste. Si aucune

commande ne correspond aux critères choisis, un message indiquant « Aucune commande trouvée » est affiché.

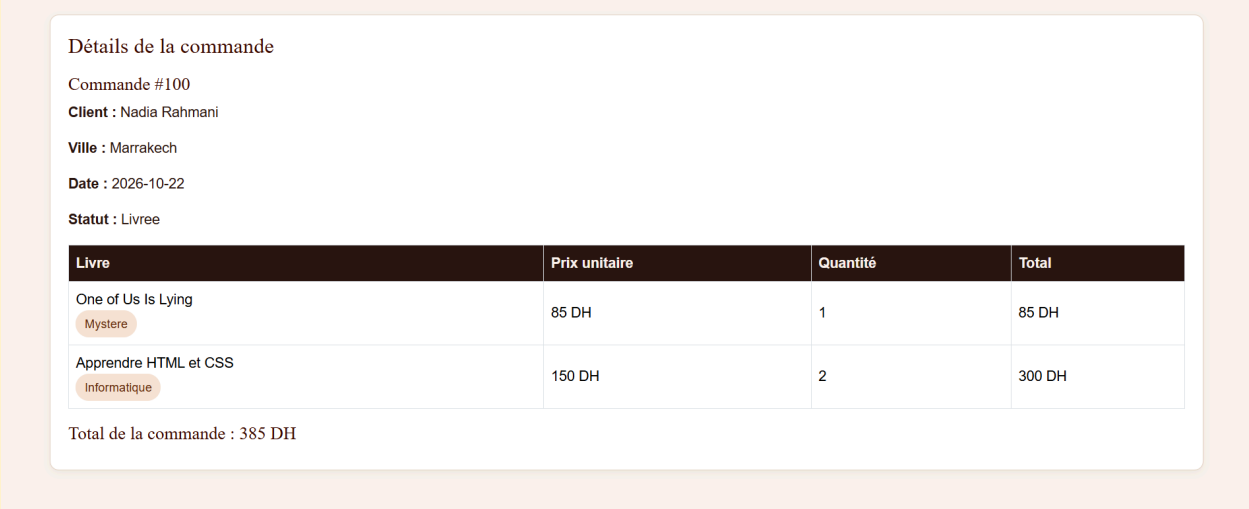


9.5. Affichage du détail d'une commande

Chaque commande affichée possède un bouton « Voir les détails ». Lorsqu'il est cliqué, un événement permet de récupérer l'identifiant de la commande sélectionnée et d'appeler la fonction `renderOrderDetails()`.

Cette fonction retrouve la commande à partir de son identifiant, puis récupère les informations du client et des livres associés. Elle affiche ensuite le nom du client, sa ville, la date, le statut et un tableau contenant les livres, les prix unitaires, les quantités et les montants des lignes. Le montant total de la commande est affiché à la fin.

Cette fonctionnalité permet de consulter les informations d'une commande sans afficher tous ses détails directement dans la liste principale.



Détails de la commande

Commande #100

Client : Nadia Rahmani

Ville : Marrakech

Date : 2026-10-22

Statut : Livree

Livre	Prix unitaire	Quantité	Total
One of Us Is Lying Mystere	85 DH	1	85 DH
Apprendre HTML et CSS Informatique	150 DH	2	300 DH

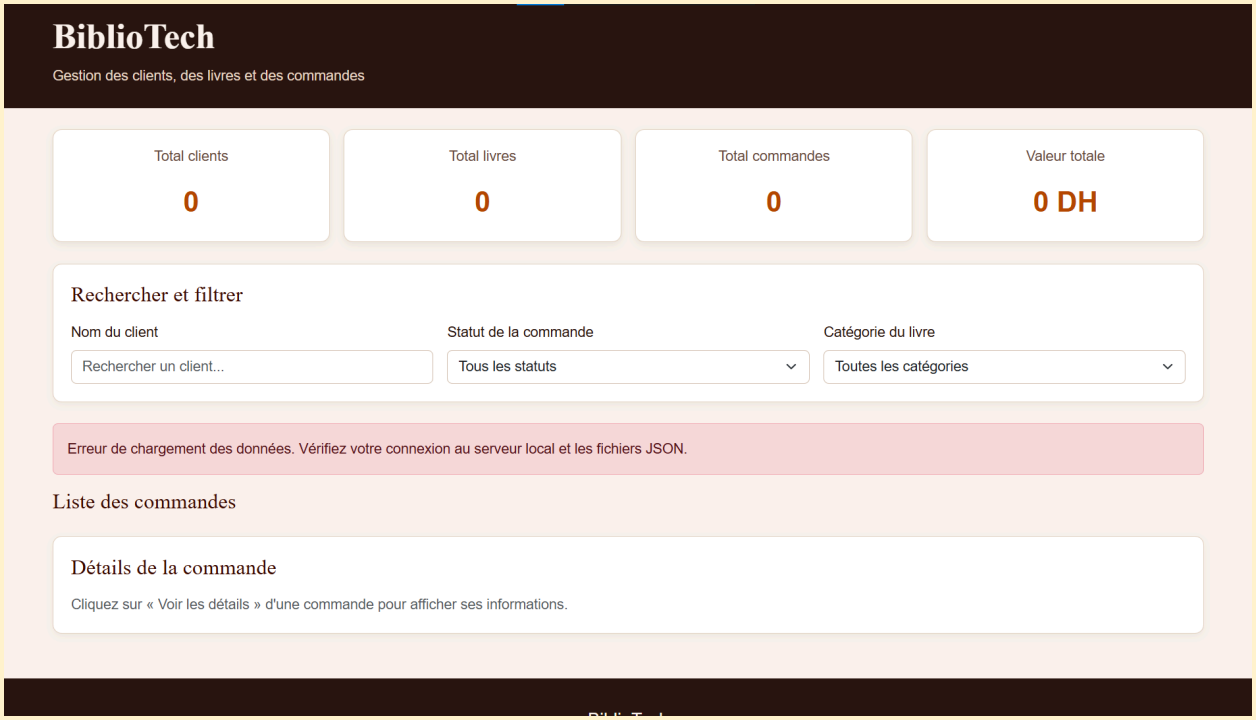
Total de la commande : 385 DH

10. Gestion des erreurs et difficulté rencontrée

Lors du développement de BiblioTech, une difficulté a été rencontrée lors du chargement des données JSON. En effet, l'utilisation de `fetch()` nécessite un serveur local dans notre cas, car l'ouverture directe du fichier HTML peut empêcher le chargement correct des fichiers JSON.

Pour résoudre ce problème, nous avons exécuté le projet à l'aide d'un serveur local afin de permettre à l'application d'accéder aux fichiers `customers.json`, `products.json` et `orders.json`.

Afin de vérifier la gestion des erreurs, nous avons effectué un test en renommant temporairement le fichier `products.json`. L'application n'a alors plus pu accéder à ce fichier. L'erreur a été détectée grâce à la vérification de la réponse de `fetch()` et à l'utilisation de `try...catch`. Un message d'erreur a ensuite été affiché pour signaler que les données n'avaient pas pu être chargées.



Après ce test, nous avons remis le fichier à son nom d'origine et vérifié que l'application fonctionnait de nouveau correctement.

Ce test nous a permis de comprendre l'importance de la gestion des erreurs pour éviter que l'application échoue silencieusement lorsqu'un fichier JSON est indisponible.

11. Utilisation de Git et GitHub

Git et GitHub ont été utilisés pour suivre les modifications du projet et publier son code en ligne. Git permet de conserver un historique des modifications, tandis que GitHub permet d'héberger le dépôt et de le partager publiquement.

11.1. Création du dépôt GitHub

Nous avons créé un dépôt public sur GitHub nommé `TP4_BiblioTech`. Ce dépôt contient les fichiers HTML, CSS,

JavaScript, les données JSON et le fichier README.md du projet.

11.2. Initialisation du dépôt local

Nous avons ouvert le terminal dans le dossier du projet, puis initialisé Git avec la commande suivante : `git init,` cette commande permet de commencer à suivre le projet avec Git. Nous avons ensuite ajouté les fichiers et créé le premier commit.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git init
Initialized empty Git repository in C:/Users/hp/Desktop/LST/Full Stack/TP4_BiblioTech/.git/
```

11.3. Ajout du dépôt distant et publication

Après la création du dépôt sur GitHub, nous avons relié le projet local au dépôt distant avec les commandes suivantes :

- `git branch -M main` : définit le nom de la branche principale à `main`.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git branch -M main
```

- `git remote add origin https://github.com/fatemaezzahra/TP4_BiblioTech.git` : associe le projet local au dépôt GitHub.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git remote add origin https://github.com/fatemaezzahra/TP4_BiblioTech.
git
```

- `git push -u origin main` : envoie les commits locaux vers GitHub et associe la branche locale à la branche distante.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 4 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (16/16), 6.80 KiB | 580.00 KiB/s, done.
Total 16 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/fatemaezzahra/TP4_BiblioTech.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Par la suite, la commande `git push` permet de publier les nouveaux commits sur GitHub.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 522 bytes | 522.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/fatemaezzahra/TP4_BiblioTech.git
   492d170..eaacc34  main -> main
```

11.4. Rôle des principales commandes Git

- `git add` : prépare les fichiers modifiés pour le prochain commit.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git add .
```

- `git commit` : enregistre les modifications préparées dans l'historique local avec un message descriptif.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git commit
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

- `git push` : envoie les commits vers le dépôt distant sur GitHub.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 522 bytes | 522.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/fatemaezzahra/TP4_BiblioTech.git
   492d170..eaacc34  main -> main
```

- `git status` : affiche l'état des fichiers du dépôt.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

- `git log --oneline` : affiche l'historique des commits sous une forme courte.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git log --oneline
be73bc1 (HEAD -> main, origin/main) Ameliorer le message de gestion des erreurs
3475de3 Centraliser le calcul du montant total
eaacc34 Complete project README
492d170 Initialisation du projet BiblioTech
```

11.5. Historique des commits

Pendant le travail, nous avons créé quatre commits enregistrant les étapes et les modifications du projet :

1. **Initialisation du projet BiblioTech** (`492d170`) : création du premier état enregistré du projet.
2. **Complete project README** (`eaacc34`) : ajout et mise à jour du fichier README.md.
3. **Centraliser le calcul du montant total** (`3475de3`) : réutilisation de la fonction `getTotalOrderValue()` dans `view.js` afin d'éviter de répéter le calcul.
4. **Ameliorer le message de gestion des erreurs** (`be73bc1`) : modification du message affiché lorsque les données ne peuvent pas être chargées.

```
C:\Users\hp\Desktop\LST\Full Stack\TP4_BiblioTech>git log --oneline
be73bc1 (HEAD -> main, origin/main) Ameliorer le message de gestion des erreurs
3475de3 Centraliser le calcul du montant total
eaacc34 Complete project README
492d170 Initialisation du projet BiblioTech
```

Ces commits permettent de consulter les différentes modifications enregistrées dans le dépôt. Le dernier état du projet a été envoyé sur GitHub.

11.6. Rôle du fichier README.md

Le fichier `README.md` présente le projet et permet aux personnes qui consultent le dépôt de comprendre son objectif et son organisation. Il contient une brève présentation de BiblioTech, la liste des fonctionnalités, les technologies utilisées, la structure des dossiers et

les instructions de lancement du site avec un serveur local.

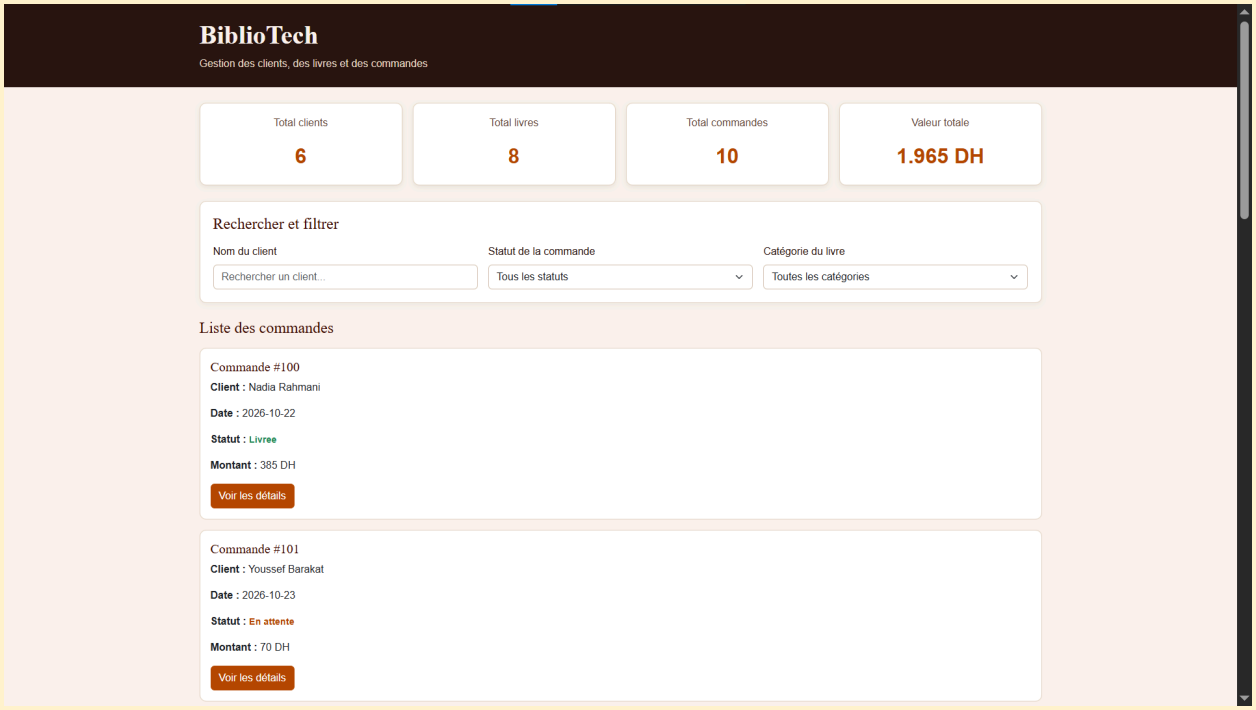
12. Conclusion

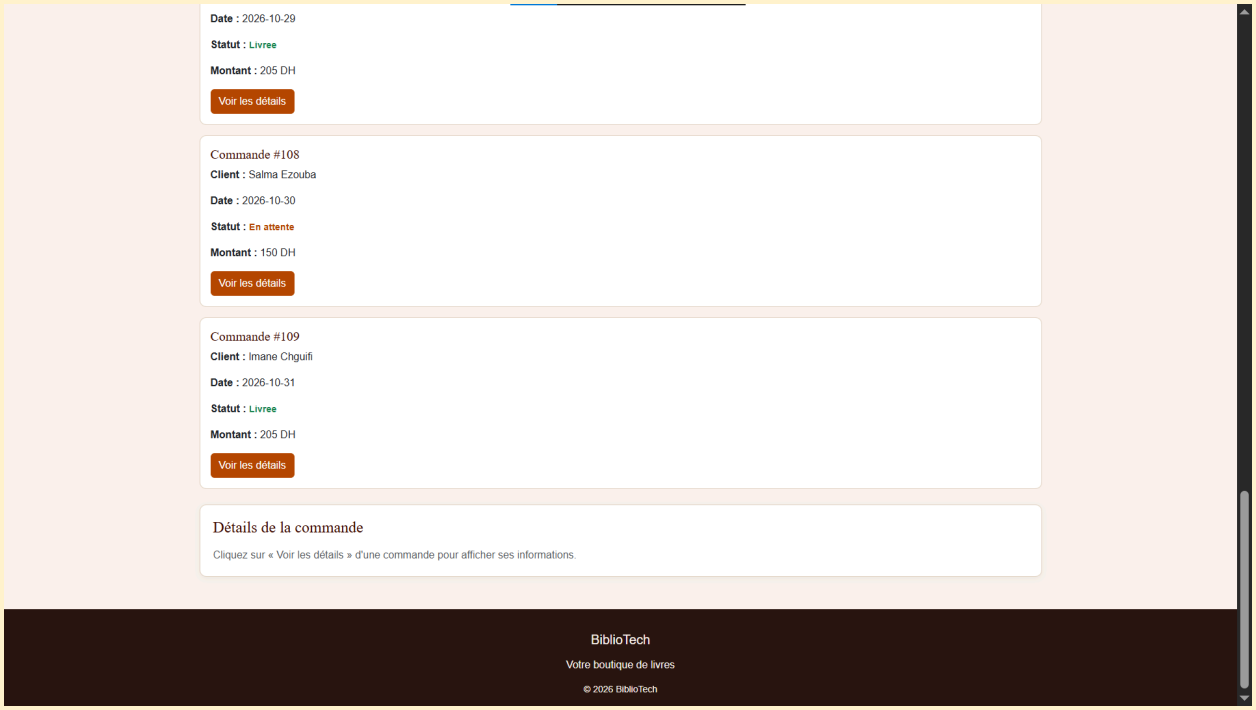
Ce TP nous a permis de réaliser une application web de gestion des commandes d'une boutique de livres. Nous avons utilisé HTML, CSS, Bootstrap, JavaScript et des fichiers JSON pour afficher les données des clients, des livres et des commandes.

Ce travail nous a également permis de pratiquer le chargement des données avec `fetch()`, l'utilisation des modules JavaScript, la recherche, les filtres et le calcul des montants. Nous avons aussi appris à gérer les erreurs et à organiser le code en plusieurs fichiers.

Enfin, l'utilisation de Git et GitHub nous a permis de suivre les modifications du projet et de publier sa version finale dans un dépôt public.

Ce TP nous a donc aidés à mieux comprendre comment organiser et développer une application web en utilisant plusieurs technologies complémentaires.


BiblioTech
Gestion des clients, des livres et des commandes
Total clients
6
Total livres
8
Total commandes
10
Valeur totale
1.965 DH
Rechercher et filtrer
Nom du client
Rechercher un client...
Statut de la commande
Tous les statuts
Catégorie du livre
Toutes les catégories
Liste des commandes
Commande #100
Client : Nadia Rahmani
Date : 2026-10-22
Statut : Livree
Montant : 385 DH
Voir les détails
Commande #101
Client : Youssef Barakat
Date : 2026-10-23
Statut : En attente
Montant : 70 DH
Voir les détails


Date : 2026-10-29
Statut : Livree
Montant : 205 DH
Voir les détails
Commande #108
Client : Salma Ezouba
Date : 2026-10-30
Statut : En attente
Montant : 150 DH
Voir les détails
Commande #109
Client : Imane Chguifi
Date : 2026-10-31
Statut : Livree
Montant : 205 DH
Voir les détails
Détails de la commande
Cliquez sur « Voir les détails » d'une commande pour afficher ses informations.
BiblioTech
Votre boutique de livres
© 2026 BiblioTech